

 Deepchecks NLP - Validation MLOps pour Sentiment Tunisien

 Vue d'ensemble

Ce document décrit l'implémentation complète de **Deepchecks NLP** pour la validation automatique du pipeline MLOps de classification de sentiment en arabe tunisien. Deepchecks NLP fournit des suites de tests automatisées pour garantir la qualité des données textuelles, détecter le drift et évaluer les performances des modèles.

 Structure des Notebooks

Le processus de validation est divisé en **3 niveaux** correspondant à **3 notebooks spécialisés** :

```
testing/
├── deepchecks_integrity.ipynb      # NIVEAU 1 : Intégrité des données
├── deepchecks_distribution.ipynb   # NIVEAU 2 : Drift et distribution
├── deepchecks_performance.ipynb   # NIVEAU 3 : Performance du modèle
└── DEEPCHECKS_NLP_DOCUMENTATION.md # Ce fichier
```

 NIVEAU 1 : Intégrité des Données Textuelles

Notebook: `deepchecks_integrity.ipynb`

Objectif

Vérifier la qualité et l'intégrité des données textuelles brutes avant l'entraînement.

Suite Utilisée

```
from deepchecks.nlp.suites import data_integrity

integrity_suite = data_integrity()
result = integrity_suite.run(train_data, test_data)
```

Checks Exécutés

Check	Description	Critère de Validation
Text Property Outliers	Détecte les textes avec des propriétés anormales (longueur, mots rares)	Pas de valeurs extrêmes

Check	Description	Critère de Validation
Unknown Tokens	Identifie les tokens jamais vus dans le vocabulaire	< 5% de tokens inconnus
Text Duplicates	Trouve les textes dupliqués dans le dataset	< 1% de duplications
Conflicting Labels	Détecte les textes identiques avec labels différents	Aucun conflit
Property Label Correlation	Vérifie si certaines propriétés sont trop corrélées aux labels	Corrélation équilibrée

Résultats Générés

-  **Rapport HTML interactif** : [deepchecks_nlp_integrity_report.html](#)
-  **Widget interactif Jupyter** : Affichage pass/fail pour chaque check
-  **Statistiques textuelles** : Longueur moyenne, nombre de mots, distribution

Exemple de Sortie

 NIVEAU 1: TEXT DATA INTEGRITY (NLP Natif)

=====

 Checks réussies: 4/5

 Statistiques Texte:
Train - Longueur moyenne: 145.3 caractères
Test - Longueur moyenne: 142.8 caractères
Train - Mots moyens: 24.7
Test - Mots moyens: 24.1

Utilisation

```
# Créer les TextData
train_text_data = TextData(raw_text=train_texts, label=train_labels,
                           task_type='text_classification')
test_text_data = TextData(raw_text=test_texts, label=test_labels,
                          task_type='text_classification')

# Exécuter les checks d'intégrité
integrity_result = run_text_integrity_checks(train_text_data, test_text_data)

# Afficher le widget interactif
integrity_result
```

 NIVEAU 2 : Drift et Distribution (Train-Test Validation)

Notebook: [deepchecks_distribution.ipynb](#)

Objectif

Détecter les différences de distribution entre les ensembles d'entraînement et de test (drift).

Suite Utilisée

```
from deepchecks.nlp.suites import train_test_validation

drift_suite = train_test_validation()
result = drift_suite.run(train_data, test_data)
```

Checks Exécutés

Check	Description	Critère de Validation
Label Drift	Compare la distribution des labels train vs test	KL divergence < 0.1
Property Drift	Détecte les changements dans les propriétés textuelles	Drift statistically insignificant
Text Embeddings Drift	Mesure le drift sémantique via embeddings	Cosine similarity > 0.95
Train Test Samples Mix	Vérifie qu'aucun échantillon test n'apparaît dans train	0 overlap

Propriétés Textuelles Calculées

- **Longueur du texte** (nombre de caractères)
- **Nombre de mots**
- **Taux de vocabulaire unique**
- **Sentiment lexical** (si disponible)
- **Complexité syntaxique**

Résultats Générés

-  **Rapport HTML interactif** : [deepchecks_nlp_drift_report.html](#)
-  **Widget interactif Jupyter** : Visualisations de drift pour chaque propriété
-  **Graphiques de distribution** : Comparaison train vs test

Exemple de Sortie

```
 NIVEAU 2: NLP TRAIN-TEST DRIFT
=====
```

☑ Checks réussies: 3/4

📊 Distribution des Labels:

Train:

0 0.513

1 0.487

Test:

0 0.514

1 0.486

Utilisation

```
# Calculer les propriétés textuelles
train_text_data.calculate_builtin_properties()
test_text_data.calculate_builtin_properties()

# Exécuter les checks de drift
drift_result = run_nlp_drift_checks(train_text_data, test_text_data)

# Afficher le widget interactif
drift_result
```

🏆 NIVEAU 3 : Performance du Modèle

Notebook: [deepchecks_performance.ipynb](#)

Objectif

Évaluer les performances du modèle et détecter les problèmes potentiels (overfitting, bias).

Suite Utilisée

```
from deepchecks.nlp.suites import model_evaluation

performance_suite = model_evaluation()
result = performance_suite.run(
    train_dataset=train_data,
    test_dataset=test_data,
    train_predictions=y_train_pred,
    test_predictions=y_test_pred,
    train_probabilities=y_train_proba,
    test_probabilities=y_test_proba
)
```

Checks Exécutés

Check	Description	Critère de Validation
Prediction Drift	Compare les distributions de prédictions train vs test	Drift < threshold
Train Test Performance	Compare les métriques train vs test	Gap < 10%
Property Segments Performance	Identifie les segments de mauvaise performance	All segments > 70% acc
Confusion Matrix Report	Matrice de confusion détaillée	Balanced performance

Métriques Calculées

Métriques Principales

- **Accuracy** : Précision globale
- **F1-Score** : Moyenne harmonique précision/rappel
- **Precision** : Taux de vrais positifs
- **Recall** : Taux de détection

Détection d'Overfitting

```
overfit_gap = train_accuracy - test_accuracy

if overfit_gap > 0.1:
    print("⚠ ATTENTION: Possible overfitting détecté!")
```

Résultats Générés

-  **Rapport HTML interactif** : [deepchecks_nlp_performance_report.html](#)
-  **Widget interactif Jupyter** : Métriques visuelles et segments de performance
-  **Classification Report** : Précision par classe
-  **Confusion Matrix** : Matrice de confusion

Exemple de Sortie

 Métriques du Modèle:
Train Accuracy: 0.8552
Train F1: 0.8422
Test Accuracy: 0.7364
Test F1: 0.7056
Test Precision: 0.7725
Test Recall: 0.6494

 Écart Train/Test: 0.1187

⚠ ATTENTION: Possible overfitting détecté!

📊 Matrice de Confusion (Test):
[[217 48]
[88 163]]

Utilisation

```
# Générer les prédictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
y_train_proba = model.predict_proba(X_train)
y_test_proba = model.predict_proba(X_test)

# Exécuter les checks de performance
performance_result, metrics = run_nlp_model_performance(
    model, train_text_data, test_text_data, X_train, X_test
)

# Afficher le widget interactif
performance_result
```

🔧 Configuration Technique

Environnement

```
conda create -n deepchecks-nlp python=3.9 -y
conda activate deepchecks-nlp
pip install deepchecks[nlp] transformers sentencepiece
```

Dépendances Clés

```
deepchecks==0.19.x
transformers>=4.30.0
sentencepiece>=0.1.99
torch>=2.0.0
scikit-learn>=1.3.0
pandas>=2.0.0
numpy>=1.24.0
```

Structure des Données

Format d'Entrée

```
# Textes nettoyés
texts = ["نص عربي تونسي", "texte tunisien", ...]
labels = ['0', '1', '0', ...] # String labels

# Créer TextData Deepchecks
text_data = TextData(
    raw_text=texts,
    label=labels,
    task_type='text_classification',
    name='dataset_name'
)
```

Format des Prédictions

```
# Prédictions alignées avec les labels
y_pred = np.array(['0', '1', '0', ...]) # Same type as labels
y_proba = np.array([[0.8, 0.2], [0.3, 0.7], ...]) # Probabilities
```

Rapports et Visualisations

Rapports HTML Générés

1. `deepchecks_nlp_integrity_report.html`

- Suite d'intégrité complète
- Détails de chaque check
- Graphiques interactifs

2. `deepchecks_nlp_drift_report.html`

- Visualisations de drift
- Comparaisons de distributions
- Analyses de propriétés

3. `deepchecks_nlp_performance_report.html`

- Métriques de performance
- Analyse de segments
- Matrice de confusion interactive

Widgets Interactifs Jupyter

Chaque niveau génère un widget interactif permettant :

- ☒ Voir les checks qui ont réussi
- ☒ Voir les checks qui ont échoué
-  Voir les warnings

-  Explorer les visualisations détaillées

Workflow d'Exécution

1. Préparation des Données

```
# Charger les données préprocessées
data = load_preprocessed_data()
texts, labels = load_cleaned_texts()

# Créer le split train/test
train_df, test_df = train_test_split(df, test_size=0.15, random_state=42)
```

2. Exécution Séquentielle

```
# NIVEAU 1: Intégrité
integrity_result = run_text_integrity_checks(train_data, test_data)
integrity_result.save_as_html('integrity_report.html')

# NIVEAU 2: Drift
drift_result = run_nlp_drift_checks(train_data, test_data)
drift_result.save_as_html('drift_report.html')

# NIVEAU 3: Performance
performance_result, metrics = run_nlp_model_performance(
    model, train_data, test_data, X_train, X_test
)
performance_result.save_as_html('performance_report.html')
```

3. Analyse des Résultats

```
# Vérifier les checks réussis
for check_result in integrity_result.results:
    if hasattr(check_result, 'passed_conditions'):
        if check_result.passed_conditions():
            print(f"✅ {check_result.get_header()}")
        else:
            print(f"❌ {check_result.get_header()}")
```

Recommandations et Actions

Si Overfitting Détecté (gap > 10%)

- ☒ Augmenter les données d'entraînement

- ☒ Appliquer une régularisation plus forte
- ☒ Réduire la complexité du modèle
- ☒ Utiliser data augmentation

Si F1-Score Faible (< 70%)

- ☒ Ajouter des features NLP (n-grams, embeddings)
- ☒ Fine-tuner TunBERT sur le domaine
- ☒ Améliorer le nettoyage des données
- ☒ Équilibrer les classes

Si Drift Détecté

- ☒ Vérifier la cohérence du split train/test
- ☒ Analyser les causes du drift
- ☒ Ré-échantillonner les données
- ☒ Mettre à jour le modèle régulièrement

Ressources

Documentation Officielle

- [Deepchecks NLP Docs](#)
- [Deepchecks Suites](#)
- [TextData API](#)

Guides Deepchecks

- [Data Integrity Suite](#)
- [Train-Test Validation Suite](#)
- [Model Evaluation Suite](#)

Notes Importantes

Différences avec Deepchecks Tabular

Aspect	Tabular (ancien)	NLP (actuel)
Import	<code>deepchecks.tabular</code>	<code>deepchecks.nlp</code>
Classe de données	<code>Dataset(df)</code>	<code>TextData(raw_text=...)</code>
Input	Features numériques	Texte brut
Analyse	Statistiques colonnes	Analyse sémantique
Drift	Feature drift	Property drift + embeddings

Limitations Connues

1. **Property Calculation** : Nécessite `calculate_builtin_properties()` avant les checks de drift
 2. **Metadata** : Certains checks nécessitent des métadonnées (non implémenté ici)
 3. **Embeddings** : Les checks d'embeddings peuvent être lents sur de grands datasets
 4. **Predictions Format** : Les prédictions doivent être passées directement à `.run()`, pas comme attributs
-

☒ Checklist de Validation

- ☐ NIVEAU 1 exécuté sans erreur
 - ☐ NIVEAU 2 exécuté sans erreur
 - ☐ NIVEAU 3 exécuté sans erreur
 - ☐ Rapports HTML générés
 - ☐ Aucun check critique échoué
 - ☐ Overfitting < 10%
 - ☐ F1-Score > 70%
 - ☐ Drift acceptable
 - ☐ Documentation à jour
-

Version: 1.0

Date: 15 décembre 2025

Auteur: MLOps Election Team

Framework: Deepchecks NLP 0.19.x